체크박스(☐)를 클릭하면 체크되고, Edge·크롬·Adobe Reader 에서 저장(Ctrl+S)하면 남습니다.

오류·수정·변경 보고 체계와 팀 운영 규칙

2026-10-06 · 작성 Sung Hyung Lee (PM) · 6인 팀 공용 · 적용: 교과 7 조각김치 이물검출 라벨링 프로젝트

한눈에 보기

클래스에서는 말로 바로 보고하고, 기록은 서기가 합니다. 서기의 하루 기록은 PM 이 읽고 확인한 뒤에 확정합니다. 집에서는 GitHub Issue 로 먼저 보고합니다. 모든 보고는 PM 이 1차로 받고, 담당자가 고치고, QA 가 확인한 뒤, PM 이 최종 승인합니다.

항목	핵심
프로젝트 시간	시작 **10:00 또는 11:00** (오전 수업이 9:50 또는 10:50 에 끝남) ~ **18:00**, 점심 **13:00~14:00**, 매시 50분부터 10분 휴식
보고 흐름	발견 → PM 1차 보고 → 담당자 상의·수정 → QA 확인 → PM 최종 승인
수정본 배포	마감 때 병합 → 다음 날 시작 회의 때 배포 (긴급만 즉시)
기록	**서기**가 보고 내용을 그때그때 메모 → 마감 때 하루 기록 작성 → **PM 검토·확인** → 서기가 GitHub Issue + `reports/test_report.md` 에 올림

1. 하루 시간표

오전 수업은 9:00 에 시작해 **9:50** 또는 **10:50** 에 끝나므로, 프로젝트는 그날에 따라 **10:00** 또는 **11:00** 에 시작합니다. 수업과 같이 매시 **50분~정각**은 휴식입니다.

시간	내용	주관
09:00 ~ 09:50 / 10:50	오전 수업	-
10:00 또는 11:00 시작 회의 (10분)	어제 수정 내용 공유, 새 버전 배포(`bash update.sh`), 병합본 WORK 배포, 오늘 라벨링 목표 공지	PM
시작 회의 ~ 12:50	라벨링 담당자 라벨링 진행 / 문제를 발견하면 누구든 바로 PM 에게 말로 보고	라벨링 담당자 · 전원
12:40 ~ 12:50 오전 정리 (10분)	서기가 오전 메모를 읽어 줌 → PM 이 등급·담당자 확정, 오후 수정 순서 결정 → 서기가 결정 내용 메모	PM · 서기
13:00 ~ 14:00	점심시간	-
14:00 ~ 17:20	라벨링 담당자 라벨링 계속 / 수정 담당자 수정 → QA 확인	담당자 · QA
17:20 ~ 17:35 마감 루틴 ① (15분)	각자 WORK 를 공유폴더로 복사 → 병합(`04_merge.py`) → Validation → QA Summary / 그동안 서기는 하루 기록 작성	DOC · QA · 서기
17:35 ~ 17:50 마감 루틴 ② (15분)	**서기가 하루 기록을 PM 에게 보고 → PM 검토·수정·확인** → 개선 제안 결정 → 수정본 main 병합 → 서기가 확정본을 Issue· `test_report.md` 에 올림	PM · 서기

17:50 ~ 18:00	다음 날 목표·배포 예정 수정 사항 공지	PM

- 매시 50분~정각 휴식 10분에는 보고·상의를 미루지 않아도 되지만, **긴급 건은 휴식과 상관없이 즉시** 처리합니다.
- 시작이 11:00 인 날은 오전 작업이 짧으므로 오전 정리(12:40)는 5분으로 줄여도 됩니다.

2. 보고 흐름 (클래스 시간)

단계	누가	무엇을
①	발견한 사람	그 자리에서 **PM 에게 말로 보고** + 화면 캡처 한 장 (연습 데이터 화면, 실제 김치 사진이 보이면 안 됨)
②	PM (+ 서기)	PM 이 바로 판단: **등급**(긴급·일반·개선)과 **담당 역할** 지정. **서기**가 옆에서 듣고 한 줄 메모 (PM 은 따로 적지 않음)
③	담당자 (+ 발견자)	그 자리에서 상의 → 담당자가 브랜치에서 수정
④	QA	수정한 자리에서 바로 Retest (같은 순서로 다시 해 보고 통과 확인) → 결과를 서기에게 알림
⑤	PM	최종 확인·승인 → 마감 루틴에서 main 병합 → 다음 날 시작 회의에서 배포·공지 (서기 기록에 승인 표시)

문제 종류별 담당 역할

문제 종류	담당 역할	주로 보는 코드
화면·메뉴·목록·버튼·단축키	GUI	`src/ui/`
BBox 위치·크기, Zoom/Pan, 좌표 어긋남	COORD	`src/bbox/view.py`, `canvas_editor.py`, `yolo_io.py` 변환
저장·불러오기, 라벨 유실, 폴더·경로	STORAGE	`src/bbox/session.py`, `yolo/yolo_io.py`, `yolo/dataset.py`
Validation 결과, 검수 기준, 테스트	QA	`src/validation/validator.py`, 스크립트 00·02·06·09
Manifest·병합·통계·문서	DOC	`src/manifest.py`, `validation/stats.py`, 스크립트 01·04·05·08
설치·실행 환경, Git, 일정, FINAL	PM	`setup_wsl.sh`, `run.sh`, `config.py`, 스크립트 03·07

3. 등급별 처리

등급	예시	클래스에서	집에서
긴급	저장 안 됨, 라벨 유실, 프로그램 꺼짐, RAW 가 바뀜	바로 PM 과 담당자를 **함께** 불러 확인. 해당 작업은 멈추고 즉시 수정·배포	카톡으로 PM·담당자에게 동시에 알린 뒤 Issue 등록

일반	기능 오류, 잘못된 표시, 계산이 맞지 않음	PM 에게 말로 보고 → 담당자가 그날 오후 중 수정	Issue 등록, 다음 날 시작 회의에서 배정
개선	불편함, UI 제안, 새 기능 아이디어	서기에게 말하면 서기가 개선 목록에 적음 → 마감 루틴에서 PM 이 결정	Issue 등록 (라벨 `개선`)

4. 수정본 배포 시점

- 라벨링 중간에 프로그램을 자주 바꾸면 작업이 끊기고 팀원마다 버전이 달라집니다.
- 그래서 수정본은 **마감 루틴에서 main 에 병합**하고, **다음 날 시작 회의에서 한 번에 배포**합니다.
- **긴급 건만 예외**로 고친 즉시 배포하고, 전원이 그 자리에서 `bash update.sh` 를 실행합니다.
- 배포 때는 PM 이 "무엇이 바뀌었는지"를 한 줄씩 공지합니다.

5. 기록: 서기가 쓰고 PM 이 확인

역할 나누기

누가	하는 일
서기	보고·결정·수정·Retest 결과를 그때그때 메모 → 마감 때 하루 기록 작성 → PM 확인 후 GitHub Issue 와 `reports/test_report.md` 에 올림
PM	직접 적지 않음. 서기 기록을 **읽고 빠진 것·틀린 것·결정 사항을 바로잡은 뒤 확인** (확인 전에는 확정 아님)
발견자·담당자·QA	보고·수정·Retest 결과를 PM 과 서기가 들을 수 있게 말로 알림

PM 이 내용을 알게 되는 세 번의 시점

1. **보고 순간**: 모든 보고는 PM 이 1차로 받으므로 실시간으로 압니다. 서기는 옆에서 메모만 합니다.
2. **12:40 오전 정리**: 서기가 오전 메모를 읽어 주고, PM 이 빠진 것 없이 등급·담당을 확정합니다.
3. **17:35 마감 보고**: 서기가 하루 기록 전체를 PM 에게 보고하고, PM 이 검토·확인합니다. **PM 확인이 끝난 기록만** Issue·`test_report.md` 에 올립니다.

하루 기록 양식 (서기 작성, PM 확인)

```
날짜 · 발견자 · 등급 · 현상 · 담당 · 수정 커밋 · Retest 결과 · PM 확인
# 예
10/08 · QA · 일반 · 저장 후 목록 표시가 안 바뀜 · GUI · fix: 저장 후 목록 갱신 (#5) · 통과 · 확인

# 맨 아래 한 줄: 오늘 결정 사항 (개선 제안 반영 여부 등)
결정 · 개선 #7 이슈 칸 코드 선택식 → 보류 (PM)
```

- 기록 파일은 옵시디언 노트(날짜별) 또는 공유 문서에 쓰고, 확정본만 GitHub 에 올립니다.
- 말로만 끝나고 기록이 남지 않으면 발표 때 보여 줄 근거가 없습니다. 이 기록이 **산출물 9번(Test Report)**과 **2번(Git 이력)**의 근거입니다.

6. 집에서 작업할 때

- 보고는 **GitHub 저장소 → Issues → New issue** 로 먼저 등록합니다. PM 에게 알림이 자동으로 갑니다.

- 서기는 밤사이 올라온 Issue 를 다음 날 하루 기록에 옮겨 적고, 시작 회의에서 PM 이 함께 확인합니다.
- 집에서 고친 코드도 바로 main 에 넣지 않습니다. 브랜치에 올려 두고 **다음 날 시작 회의에서 같이 확인 후 병합**합니다.

Issue 작성 양식

```
[등급] 긴급 / 일반 / 개선
[담당 역할] GUI / COORD / STORAGE / QA / DOC / PM
[현상] 무엇을 했을 때 어떻게 되는지
[재현 순서] 1) …  2) …  3) …
[기대 결과] 원래 어떻게 되어야 하는지
[첨부] 화면 캡처 (실제 김치 사진이 보이면 안 됨 → 연습 데이터 화면으로)
```

7. 지켜야 할 규칙

☐ **라벨링이 1순위.** 라벨링은 배정된 담당자가 진행합니다.

☐ **코드는 담당자만 수정.** 다른 팀원은 버그를 발견해도 직접 고치지 않고 보고합니다.

☐ **모든 보고는 PM 을 거칩니다.** 담당자에게만 말하고 끝내지 않습니다 (긴급은 PM·담당자 동시).

☐ **기록은 서기, 확정은 PM.** PM 이 확인하지 않은 기록은 GitHub·`test_report.md` 에 올리지 않습니다.

☐ **QA 확인 없이 완료 처리하지 않습니다.** "고쳤습니다"만으로 끝내지 않고 Retest 통과 후 PM 에게 최종 보고합니다.

☐ **10/7 Feature Freeze 이후에는 버그 수정만.** 새 기능은 "개선"으로 모아 PM 이 결정합니다.

☐ **수정 하나 = 커밋 하나**, 메시지에 Issue 번호를 적습니다. 예: `fix: 저장 후 목록 갱신 오류 (#5)`

☐ **main 에 직접 push 하지 않습니다.** 브랜치 → PR → PM 승인 후 병합

☐ **수정본 배포는 시작 회의 때만** (긴급 제외). 각자 마음대로 업데이트하지 않습니다.

☐ **실제 데이터는 어디에도 올리지 않습니다.** JPG·TXT·WORK·FINAL 은 Git·카톡·개인 클라우드 금지 (NDA·저작권). 캡처도 연습 데이터 화면만

☐ **RAW 는 절대 수정하지 않습니다.** 이상한 이미지는 지우지 않고 REVIEW + 사유 코드로 남깁니다.

☐ **애매하면 추측하지 않습니다.** REVIEW 로 보내고 2명이 확인한 뒤 Note 에 근거를 남깁니다.

☐ **프로그램 실행 중에는 manifest.csv 를 엑셀로 열지 않습니다** (파일이 잠겨 저장 실패).

8. 매일 체크리스트

PM

☐ 시작 회의: 어제 수정 내용 공유, 새 버전·병합본 배포, 오늘 목표 공지

☐ 12:40 오전 정리: 서기 메모 듣고 문제 등급·담당 확정

☐ 17:35 서기 하루 기록 검토·확인

☐ 마감 루틴: 수정본 승인·main 병합, 개선 제안 결정

☐ 17:50 다음 날 목표·배포 예정 공지

서기

☐ 보고·결정·수정·Retest 결과를 그때그때 메모

☐ 12:40 오전 메모를 PM 에게 읽어 주기

☐ 17:20 하루 기록 작성 → 17:35 PM 에게 보고

☐ PM 확인 후 GitHub Issue·`test_report.md` 에 올리기

담당자 · QA · DOC

☐ 담당자: 배정된 문제를 브랜치에서 수정 → QA 에 확인 요청

☐ QA: 수정 건 Retest 후 결과를 서기에게 알림, 마감 Validation·RAW 무변경 확인

☐ DOC: 병합(`04_merge.py`), QA Summary

전원

☐ 시작 회의 후 `bash update.sh` 로 같은 버전 맞추기

☐ 라벨링 담당자: 오늘 목표만큼 라벨링 진행

☐ 17:20 프로그램 종료(저장 확인) 후 내 WORK 를 공유폴더로 복사